

Overview

The CloudNine system is modeled as four integrated modules: Reservation & Scheduling, Deployment & Usage, Infrastructure, and Billing. Each module captures a distinct part of the system while remaining connected through key entities such as client, reservation, deployment, resource, and usage_record. The overall design follows a logical flow from client request → deployment → resource usage → billing, ensuring traceability across the system.

Reservation & Scheduling – Bolin Yang

Key Design Decisions

The reservation subsystem captures how clients request services and how staff manage those requests. The reservation table stores core request information such as time window, type, status, and emergency level. The need table separates resource requirements from reservations, allowing each reservation to specify multiple resource needs with attributes such as quantity, capacity requirement, and flexibility level. The actual_assignment table records how reservations are fulfilled in practice, enabling comparison between requested and assigned regions. This supports evaluation of service quality.

Assumptions

Each reservation is made by one client and handled by one staff member. A reservation may include multiple resource needs. Clients are assumed to provide a single email and phone number and can represent individuals, groups, or organizations.

Tradeoffs and Limitations

The model simplifies real world complexity by not allowing multiple clients or staff per reservation. Region preference modeling was simplified to a single requested region rather than a full preference ranking system to reduce complexity.

Handling Ambiguities

Ambiguities in how reservations turn into actual service delivery were resolved by introducing actual_assignment, which captures real outcomes without modifying the original request.

Deployment & Usage – Jack

Key Design Decisions

The deployment subsystem centers on the deployment entity, representing active workloads derived from reservations. Because deployments may involve multiple resources, the deployment_resource table models this as a many-to-many relationship. Usage is captured in usage_record, which tracks resource consumption over time. Operational behavior is recorded through deployment_event, which unifies actions such as access, scaling, migration, and updates into a single event structure.

Lookup tables such as `deployment_status` and `priority_level` standardize system states and priorities, while `deployment_priority_history` tracks how priorities change over time.

Assumptions

A reservation may produce multiple deployments, and a deployment may use multiple resources. System actions may be triggered either by staff or clients, and events are recorded accordingly.

Tradeoffs and Limitations

Event tracking is simplified into a single table rather than multiple specialized tables, improving clarity but reducing specificity. Deployment location is not stored directly and must be inferred from associated resources.

Handling Ambiguities

The relationship between deployments and infrastructure was interpreted as many-to-many due to the distributed nature of cloud workloads. Event responsibility was generalized across staff and client roles.

Infrastructure – Ella

Key Design Decisions

The infrastructure module models CloudNine's physical structure as a hierarchy: `cloud_region` → `data_center` → `availability_zone` → `resource`. This reflects how real cloud systems organize infrastructure geographically and logically.

The resource table represents deployable infrastructure components and serves as the integration point for other modules. `resource_type` classifies resources, while `resource_capacity` and `resource_availability_event` capture operational characteristics such as capabilities and downtime.

Assumptions

Each data center belongs to one region, each availability zone belongs to one data center, and each resource belongs to one availability zone and one resource type. Resource labels are assumed to be unique.

Tradeoffs and Limitations

The model does not include detailed hardware components such as individual servers or processors. Instead, resources are represented at a higher level. Relationships between resources and detailed capacity interactions are simplified and not fully modeled.

Handling Ambiguities

Details about the physical infrastructure were not fully specified. A standard cloud structure (`region` → `data center` → `availability zone` → `resource`) was used to organize the system. The focus was placed on how resources are used rather than how they are physically built.

Billing – Katie

Key Design Decisions

The billing system distinguishes between clients, beneficiaries, and paying parties to support flexible payment relationships. All entities are represented as clients, while the `paying_party` and `beneficiary` tables define their roles in billing.

The `beneficiary_paying_party` bridge table resolves the many-to-many relationship between payers and beneficiaries, allowing scenarios such as organizations paying for employees, shared cost centers, or changing payment responsibilities.

Billing is structured using:

- `bill_header` for overall invoice information
- `bill_item` for usage-based charges
- `bill_adjustment` for credits, refunds, or surcharges
- `payment` and `payment_method` for financial transactions

This design ensures that all charges are traceable back to usage through `bill_item`.

Assumptions

Any client can act as a payer or a beneficiary. Each bill is linked to one paying party, but it may include charges for multiple beneficiaries. Payment details are stored, but full financial processing is not modeled.

Tradeoffs and Limitations

Advanced billing features such as subscriptions, installment payments, and automatic pricing updates are not implemented. Adjustments like credits or fees are recorded, but their logic is not enforced by the database.

Handling Ambiguities

Payment responsibility was unclear, so payer and beneficiary roles were separated. A bridge table was used to support cases where one payer covers multiple beneficiaries or multiple payers share responsibility.